

Regulární výrazy v Pythonu

Python má zabudované regulární výrazy v knihovně `re`. Začneme tedy jejím importem.

```
import re
```

Název funkce	Znak v teorii	Znak v pythonu
Zřetězení	. nebo žádný	žádný
Seskupení	()	()
Sjednocení	+	
Iterace	*	*

Speciální případy iterace:

*	Libovolný počet ≥ 0
+	Libovolný počet > 0
.	Právě jeden výskyt
?	Maximálně jeden výskyt
{X}	X výskytů
{X,}	Minimálně X výskytů
{X,Y}	Počet výskytů je mezi X a Y

Funkce využíváme většinou v kombinaci s nějakými symboly a speciálními znaky. Zde jsem jako příklad vybrala první sloku básně Havran (Edgar Allan Poe, překlad Jaroslav Vrchlický).

Chceme vypsat všechna slova, která kdekoli obsahují složení “klep”.

```
havran = open('havran.txt', 'r').read().lower()
```

```
re.findall(r'\b\w*klep\w*\b', havran)
```

Výstup: [‘zaklepání’, ‘klepá’]

Příkaz `findall` najde všechny výrazy, obsahující posloupnost písmen “klep”. Sám o sobě by ale vypsal jen toto slovo. Proto využíváme i znaky `\b` a `\w`. Symbol `w` najde jakýkoliv znak (`*` $\rightarrow$ libovolný počet), `b` z obou stran řekne, že tento znak je na začátku i na konci.

<code>findall</code>	Najde všechny shody
<code>search</code>	Zjistí, jestli se nějaký výraz v textu vůbec nachází
<code>split</code>	Rozdělí text při každé shodě a znak/slovo odstraní
<code>sub</code>	Nahradí znak nebo skupinu znaků jiným

<code>\b</code>	Shoda na začátku nebo na konci slova
<code>\w</code>	Vrátí shodu, kde nějaké slovo obsahuje daný znak (skupinu znaků) kdekoli.
<code>\d</code>	Vrátí ty části textu, které obsahují číslice.
<code>\s</code>	Vrátí shodu obsahující bílé znaky (mezery, tabulátory)

Symboly psané velkým písmenem jsou potom negací.

Příklady použití:

Ověření správnosti emailové adresy

```
valid_email = (r'^[a-z0-9. ]*@(gmail\.com|seznam\.cz)$')
```

To znamená, že email může obsahovat malá písmena, číslice, tečky nebo podtržítka. U všech adres se vše pak změní na malá písmena, proto není potřeba uvažovat velká. Dále *zavináč* a pak *gmail.com* nebo *seznam.cz*. Vybrala jsem v rámci tohoto úkolu dvě nejčastější domény. Je potřeba to nějak omezit, a nepovolovat jakékoliv slovo. Zároveň pro takto malou úlohu není potřeba uvažovat domén více.

Můj kód pak pokračuje takto:

```
email1 = 'vi.olak3jrova@gmail.com'.lower()
email2 = 'ViOlAk.e.j.rova666@seznam.c'.lower()
email3 = '.....@GMAIL.com'.lower()

x,y,z = re.match(valid_email,email1)
re.match(valid_email,email2),re.match(valid_email,email3)

emails = [x, y, z]

for i, result in enumerate(emails):

    if result is None:
        print(f"email{i} není validní")

    else:
        print(f"email {i} je v pořádku")
```

Vyhledávání údajů v textovém souboru

Další využití regulárních výrazů je například při hledání konkrétních vzorů v textu. Pomocí nich lze z textového souboru vyhledat například všechny telefonní čísla, e-mailové adresy nebo webové stránky.

Nejprve definujeme formát těchto údajů a poté vyhledáváme pomocí `re.match()`

```
valid_email = (r'^[a-z0-9. ]*@(gmail.com|seznam.cz)$')
valid_telefon = (r'^(\+420)?[0-9]{9}$')
valid_web = (r'^(https://)(www.)?[a-zA-Z0-9]*.(com|cz|eu|org)$')
```

Vytvoříme prázdné seznamy *email*, *telefon*, *web* a *ostatní*, kam budeme nalezené údaje ukládat. Seznam *ostatní* slouží k tomu, abychom našli případné chybně zapsané údaje, a neztratili je.

```
for x in soubor:

    if re.match(valid_email, x):
```

```
maily.append(x)
```

atd.

(celý kód je přiložen)

Regulární výrazy v grepu

Příkaz grep slouží v systému Linux k vyhledávání řetězců v souborech podle zadaného vzoru. Zde budu používat terminál počítače s Ubuntu.

Symbole regulárních výrazů jsou stejné jako v Pythonu, až na to, že grep je citlivý na speciální znaky. Je tedy třeba před každý z nich vložit zpětné lomítko \, kterým říkáme, aby následující znak bral jako obyčejný text a ignoroval jeho význam.

Druhá možnost je, využít verzi grep -E, kde to potřeba není.

Příklady uvádím stejné jako v případě pythonu. Hledání shody v textu je zde snažší, protože stačí jen dát do uvozovek požadovaný výraz, a grep najde všechny shody.

```
viola@viola-TravelMate-P215-54:~/Dokumenty/teoreticka$ grep "klepá" havran.txt
ozval se jak zaklepání
„klepá na dveře, víc nic
```

Druhá úloha se zde řeší tak, že napíšeme vzor, podle kterého chceme hledat, a opět se nám vypíší všechny shody.

Aby byl vidět rozdíl mezi grep a grep -E, tak jsem použila pro každý z příkladů jednu možnost.

```
viola@viola-TravelMate-P215-54:~/Dokumenty/teoreticka$ grep "[a-z0-9._]\+@(\gmail\.com\|seznam\.cz\)" soubor.txt
jmeno.prijmeni@seznam.cz
georgeorwell@gmail.com
gary.yourosky@seznam.cz
scottjurek@gmail.com
emailovaadresa@seznam.cz
```

```
viola@viola-TravelMate-P215-54:~/Dokumenty/teoreticka$ grep -E "^(https://)(www\.)?[a-z0-9]+\.(com|cz|eu|org)\/?$" soubor.txt
https://stackoverflow.com/
https://soucitne.cz/
https://www.w3schools.com/
https://cubeoftruth.com
https://sledujsvectvi.cz/
https://dontwatch.org/
https://www.facebook.com/
https://www.koneclzionleku.cz/
https://www.youtube.com/
viola@viola-TravelMate-P215-54:~/Dokumenty/teoreticka$
```

Použité zdroje

Python:

https://www.w3schools.com/python/python_regex.asp

Linux grep:

<https://www.geeksforgeeks.org/linux-unix/regular-expression-grep/>

<https://linuxize.com/post/regular-expressions-in-grep/>

Úryvek z básně:
POE, Edgar Allan. Havran a jiné básně [online]. Přel. Jaroslav VRCHLICKÝ. V MKP 1. vyd. Praha:
Městská knihovna v Praze, 2013 Dostupné z:
https://web2.mlp.cz/koweb/00/03/92/94/91/havran_a_jine_basne.pdf